

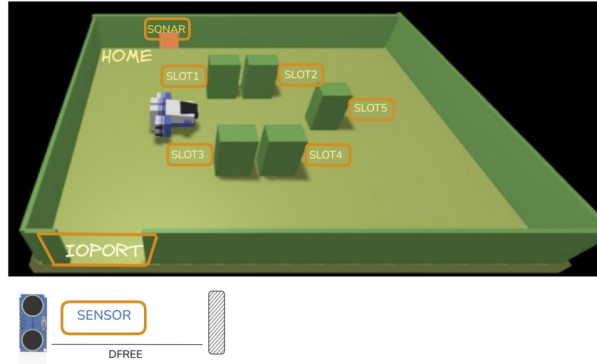
Maritime CargoService - Sprint0

Andrea Passini, Paolo Savino, Alessandra Tolomelli

Luglio 2026

1 Introduzione

Un'azienda di trasporto marittimo di merci (**committente**) intende automatizzare le operazioni di carico dei container nella stiva della nave (**hold**). A tale scopo, l'azienda prevede di impiegare un *Differential Drive Robot*, ossia il **cargorobot**. La hold è un'area rettangolare e piana dotata di una porta di ingresso/uscita (**IOPort**). L'area dispone di 4 slot per immagazzinare i container e di uno slot chiamato **slot5**.



Con riferimento alla planimetria del sistema:

- Gli **slot1-4** rappresentano le aree della stiva riservate per immagazzinare un container ciascuno.
- Lo **slot5** rappresenta un'area in cui il cargorobot deve depositare temporaneamente un container, prima di posizionarlo in uno degli slot1-4. Durante il deposito temporaneo, un dispositivo '**marker**' etichetta il container con un codice a barre identificativo e segnala quando questa attività di marcatura è completata.
- L'**IOPort** è un dispositivo dotato di un pulsante, il **pushbutton**, e di un **display**. Il pushbutton viene premuto dal cliente per inviare una richiesta di carico di un container sulla nave. Il display viene utilizzato per mostrare la risposta alla richiesta e per mostrare lo stato corrente della hold.
- Il **sensore** associato all'IOPort è un dispositivo, **sonar**, utilizzato per rilevare la presenza di un container, quando misura una distanza D tale che $D < DFREE/2$, per un tempo ragionevole (ad es. 3 secondi).

L'obiettivo dello Sprint 0 è formalizzare e disambiguare i requisiti forniti dalla committente in linguaggio naturale, costruire un primo modello del sistema, evidenziare il core business, motivare la scelta del linguaggio di modellazione e definire un primo insieme di piani di test funzionali e il goal dello Sprint 1. I requisiti del progetto sono riportati nel documento disponibile al seguente [link](#).

2 Requisiti

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Il cargoservice è in grado di ricevere una richiesta di carico di un container (**request to load**) inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.
- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.

Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**. Successivamente,

il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.

Il servizio deve inoltre mostrare sul display dell'IOPort:

- Lo stato attuale della hold.
- Il messaggio “**Service working**” quando tutto sta procedendo correttamente.
- Il messaggio “**Out of service**” se il sensore sonar misura una distanza $D > DFREE$ per almeno 3 secondi (possibile guasto del sonar).

2.1 Domande aperte al committente

1. Rilevazione dello slot libero

Cosa si intende per slot libero? Nel caso in cui ci siano più slot liberi disponibili, quale scelgo?

Risposta del committente: Per slot libero si intende uno Slot non ancora occupato da un container. Nel caso ci siano più slot disponibili viene scelto uno slot qualsiasi.

2. Durata dell'attività di marcatura

Qual è il tempo (medio o massimo) richiesto dal dispositivo marker per completare l'etichettatura del container nello slot5? È un valore fisso o variabile? In assenza di indicazioni, si assumerà un valore configurabile a scopo di test (es. 10 secondi), da confermare in seguito.

Risposta del committente: per il tempo di etichettatura è possibile assumere un valore fisso di 2 secondi.

3. Gestione del sistema durante la fase di marcatura

Mentre il container è in fase di marcatura nello slot5, cosa deve fare il cargorobot (rimanere fermo in attesa presso lo slot5, oppure tornare in posizione HOME e rientrare al termine della marcatura) e come deve comportarsi il sistema se, nel frattempo, un nuovo container viene esposto al sensore sonar dell'IOPort (ignorare il segnale, rispondere retrylater, o altro)?

Risposta del committente: Durante la fase di marcatura il cargorobot rimane fermo in attesa presso lo slot5.

4. Liberazione degli slot occupati

I requisiti descrivono il processo di carico dei container negli slot1-4, ma non specificano come venga gestita la liberazione di uno slot già occupato. È previsto un messaggio/richiesta analogo alla loadRequest per lo scarico (es. unloadRequest)? Oppure è gestito da un processo esterno al sistema? Senza questo meccanismo, il sistema risulterebbe permanentemente pieno dopo il quarto carico.

Risposta del committente: Non sono previste funzioni di scarico, perciò il sistema viene considerato permanentemente pieno

3 Analisi dei Requisiti

3.1 Scelta del Linguaggio QAK

QAK è un linguaggio utile per modellare sistemi distribuiti ed è particolarmente efficace nel rappresentare il concetto di attore autonomo e di messaggio, riducendo l'*Abstraction Gap* tra i requisiti e la loro implementazione in un contesto distribuito ed eterogeneo. Analizzando le specifiche fornite, si nota come il sistema sia caratterizzato da componenti eterogenei che devono coordinarsi nel tempo reagendo a eventi esterni: queste precise caratteristiche si traducono direttamente nei costrutti nativi del linguaggio QAK. Dal modello viene, inoltre, generato automaticamente codice Kotlin eseguibile, il che consente di disporre di un primo prototipo osservabile.

3.1.1 Formalizzazione della richiesta

L'unica richiesta che si evince è quella di carico, che viene modellata come **request** in qak. La richiesta viene inviata dal customer al cargoservice tramite l'IOPort.

```
Request load_request : loadRequest(none)
Reply load_accepted : loadAccepted(slotID) for load_request
Reply load_retrylater : loadRetryLater(none) for load_request
Reply load_refused : loadRefused(none) for load_request
```

3.2 Macro-Componenti e natura software

Il codice sorgente, comprensivo di tutti i modelli QAK e dei componenti descritti in seguito, è interamente disponibile al seguente [link](#).

3.2.1 CargoService

CargoService è il componente che gestisce le richieste di carico e verifica le condizioni coordinando gli altri componenti dell'architettura.

```
QActor cargoservice context ctxcargoservice {

  State s0 initial {
    println("cargoservice | started") color blue
  }
  Goto disengaged

  State disengaged {
    println("cargoservice | DISENGAGED: waiting for load_request") color green
  }
  Transition t0
    whenRequest load_request -> handle_load_request

  State handle_load_request {
    println("ioport -> cargoservice | load_request: loadRequest(none)") color cyan

    /* Testplan minimale:
    *
    * T1: richiesta accettata      -> il sistema deve passare a ENGAGED
    * T2: richiesta rinviata      -> il sistema deve tornare/restare DISENGAGED
    * T3: richiesta rifiutata     -> il sistema deve tornare/restare DISENGAGED
    */

    // --- T1: accettata ---
    println("cargoservice -> ioport | load_accepted: loadAccepted(slot1)")
    color green
    replyTo load_request with load_accepted : loadAccepted(slot1)

    /*
    // --- T2: rinviata ---
    println("cargoservice -> ioport | load_retrylater: loadRetryLater(none)")
    color yellow
    replyTo load_request with load_retrylater : loadRetryLater(none)

    // --- T3: rifiutata ---
    println("cargoservice -> ioport | load_refused: loadRefused(none)") color red
    replyTo load_request with load_refused : loadRefused(none)
    */
  }
}
}
```

```
// Il Goto va scelto in coerenza con il blocco reply decommentato sopra:
// - se attiva T1 (load_accepted) -> Goto engaged
// - se attiva T2 o T3 (retrylater/refused) -> Goto disengaged
Goto engaged
// Goto disengaged // <-- usare questa riga al posto di quella sopra per T2/T3

State engaged {
    println("cargoservice | ENGAGED") color green
}
```

3.2.2 CargoRobot

Il sistema richiede di modellare il CargoRobot, ossia il sottosistema responsabile degli spostamenti fisici del container nella hold. La software house ha già realizzato in precedenza un'interfaccia per utilizzare un robot DDR, ovvero [VirtualRobot26](#) o [RobotService26](#). Di seguito viene riportata una formalizzazione del suo comportamento, che si limita a ricevere comandi di spostamento e ad eseguirli.

```
QActor cargorobot context ctxrobot {
    State s0 initial {}
    Goto work
    State work {
        println("cargorobot | WORK: move container from IOPort to slot5 and then to
reserved slot") color blue
    }
}
```

3.2.3 Hold

La Hold rappresenta logicamente la stiva e può essere formalizzata come una struttura dati composta da Celle. Il componente è da sviluppare. **!!DA MODIFICARE CODICE SOTTO**

```
public enum CellType {
    FREE, OBSTACLE, HOME, SONAR, IOPORT,
    SLOT1, SLOT2, SLOT3, SLOT4, SLOT5
}
```

3.2.4 Sonar

Il Sonar rileva la presenza di un container. Dopo un'attenta riflessione con il committente, il sonar fisico è considerato fornito ed è collegato al PicoW, mentre il software di integrazione è da sviluppare.

3.2.5 IOPort

Componente responsabile dell'interazione con l'utente tramite pulsante e display. IOPort ad emette la **load_request** verso cargoservice e mostra le informazioni di stato. In questa fase l'IOPort viene modellato temporaneamente come un attore QAK autonomo per poterne simulare e validare il comportamento logico all'interno del sistema. Nelle successive fasi di progettazione e implementazione, questo attore verrà sostituito da una Web-GUI, che comunicherà con il cargoservice tramite protocolli standard.

```

QActor ioport context ctxioport {

  State s0 initial {
    println("ioport | started") color blue
  }
  Goto press_button

  State press_button {
    println("customer -> ioport | pushbutton pressed") color magenta
    println("ioport -> cargospace | sending load_request") color cyan
    request cargospace -m load_request : loadRequest(none)
  }
  Transition t0
    whenReply load_accepted -> accepted
    whenReply load_retrylater -> retrylater
    whenReply load_refused -> refused

  State accepted {
    println("ioport | DISPLAY: load accepted") color green
  }
  State retrylater {
    println("ioport | DISPLAY: retry later") color yellow
  }
  State refused {
    println("ioport | DISPLAY: load refused") color red
  }
}

```

3.2.6 Marker

Il markerdevice etichetta i container depositati nello slot5 e notifica il completamento della marcatura. Dopo una riflessione effettuata con il committente, il dispositivo è considerato disponibile in forma simulata, mentre il software di controllo è da sviluppare.

3.2.7 LED

Dopo una consultazione con il committente, si è definito che il LED è considerato un dispositivo fisico integrato nel PicoW che gestisce anche il sonar. Il software di controllo del LED è quindi da sviluppare o integrare nel software eseguito sul PicoW.

4 Piani di Test

I [TestPlan](#) validano perfettamente il flusso ideale, verificando il primo corretto passaggio a engaged e l'assegnazione iniziale dello slot. I test pronti su IOPort coprono bene la logica dell'interfaccia, controllando l'invio delle richieste e la reazione del display ai vari messaggi di risposta.

5 Goal dello Sprint 1

Il goal del prossimo sprint è quello di implementare un primo prototipo dell'architettura. Al termine dello Sprint1 il sistema dovrà essere in grado di:

- Visualizzare una prima griglia modificabile astratta per la hold e verificarne lo stato delle celle.
- Dettagliare la comunicazione tra CargoService e IOPort, gestendo gli stati di engaged e disengaged del sistema.
- Primo prototipo mock dei devices.